Introduction and lists

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Data types

- Data type system sets the stage for the capabilities of the language
- Understanding data types empowers you as a data scientist

Container sequences

- Hold other types of data
- Used for aggregation, sorting, and more
- Can be mutable (`list` , `set`) or immutable (`tuple`)
- Iterable

Lists

- Hold data in order it was added
- Mutable
- Index

Accessing single items in list

```
cookies = ['chocolate chip', 'peanut butter', 'sugar']
```

```
cookies.append('Tirggel')
```

```
print(cookies)
```

```
['chocolate chip', 'peanut butter', 'sugar', 'Tirggel']
```

```
print(cookies[2])
```

```
sugar
```

Combining lists

- Using operators, you can combine two lists into a new one

```
cakes = ['strawberry', 'vanilla']


desserts = cookies + cakes


print(desserts)
```

```
['chocolate chip', 'peanut butter', 'sugar', 'Tirggel', '
```

- `.extend()` method merges a list into another list at the end

```
cookies.extend(cakes)
```

Finding elements in a list

- `.index()` method locates the position of a data element in a list

```
position = cookies.index('sugar')


print(position)
```

```
3
```

Removing elements in a List

- `.pop()` method removes an item from a list and allows you to save it

```
name = cookies.pop(position)

print(name)
```

```
sugar
```

```
print(cookies)
```

```
['chocolate chip', 'peanut butter', 'Tirggel']
```

Iterating over lists

- **List comprehensions** are a common way of iterating over a list to perform some action on them

```
titlecase_cookies = [cookie.title() for cookie in cookies
print(titlecase_cookies)
```

```
Chocolate Chip
Peanut Butter
Tirggel
```

Sorting lists

- `sorted()` function sorts data in numerical or alphabetical order and returns a new list

```
print(cookies)
```

```
['chocolate chip', 'peanut butter', 'Tirggel']
```

```
sorted_cookies = sorted(cookies)

print(sorted_cookies)
```

```
['Tirggel', 'chocolate chip', 'peanut butter']
```

Let's practice!

DATA TYPES IN PYTHON

Meet the tuples

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Tuple, tuple

- Hold data in order
- Index
- *Immutable*
- Pairing
- Unpackable

Zipping tuples

- Tuples are commonly created by zipping lists together with `zip()`
- Two lists: `us_cookies` , `in_cookies`

```
top_pairs = list(zip(us_cookies, in_cookies))
print(top_pairs)
```

```
[('Chocolate Chip', 'Punjabi'), ('Brownies', 'Fruit Cake Rusk'),
('Peanut Butter', 'Marble Cookies'), ('Oreos', 'Kaju Pista Cookies'),
('Oatmeal Raisin', 'Almond Cookies')]
```

Unpacking tuples

- Unpacking tuples is a very expressive way for working with data

```
us_num_1, in_num_1 = top_pairs[0]
print(us_num_1)
```

```
Chocolate Chip
```

```
print(in_num_1)
```

```
Punjabi
```

More unpacking in Loops

- Unpacking is especially powerful in loops

```
for us_cookie, in_cookie in top_pairs:
    print(in_cookie)
    print(us_cookie)
```

```
Punjabi
Chocolate Chip
Fruit Cake Rusk
Brownies
# ..etc..
```

Enumerating positions

- Another useful tuple creation method is the `enumerate()` function
- Enumeration is used in loops to return the position and the data in that position while looping

```
for idx, item in enumerate(top_pairs):
    us_cookie, in_cookie = item
    print(idx, us_cookie, in_cookie)
```

```
(0, 'Chocolate Chip', 'Punjabi')
(1, 'Brownies', 'Fruit Cake Rusk')
# ..etc..
```

Be careful when making tuples

- Use `zip()`, `enumerate()`, or `()` to make tuples

```
item = ('vanilla', 'chocolate')
print(item)
```

```
('vanilla', 'chocolate')
```

- Beware of tailing commas!

```
item2 = 'butter',
print(item2)
```

```
('butter',)
```

Let's practice!

DATA TYPES IN PYTHON

Strings

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Creating formatted strings

- f-strings (formatted string literals) - `f""`

```
cookie_name = "Anzac"
cookie_price = "$1.99"

print(f"Each { cookie_name } cookie costs { cookie_price }.")
```

```
"Each Anzac cookie costs $1.99."
```

Joining with strings

- `"".join()` uses the string it's called on to join an iterable

```
child_ages = ["3", "4", "7", "8"]

print(", ".join(child_ages))
```

```
"3, 4, 7, 8"
```

```
print(f"The children are ages {','.join(child_ages[0:3])}, and {child_ages[-1]}.")
```

```
"The children are ages 3, 4, 7, and 8."
```

Matching parts of a string

- `.startswith()` and `.endswith()` methods will tell you if a string starts or ends with another character or string

```
boy_names = ["Mohamed", "Youssef", "Ahmed"]
print([name for name in boy_names if name.startswith('A')])
```

```
["Ahmed"]
```

- Be careful as these and most string functions are case-sensitive.

Searching for things in strings

- The `in` operator searches for some value in some iterable type like a string.

```
"long" in "Life is a long lesson in humility."
```

```
True
```

```
"life" in "Life is a long lesson in humility."
```

```
False
```

An approach to being case insensitive

- `.lower()` method returns a lower case string

```
"life" in "Life is a long lesson in humility.".lower()
```

```
True
```

Let's practice!

DATA TYPES IN PYTHON

Using dictionaries

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Creating and looping through dictionaries

- Hold data in key/value pairs
- Nestable (use a dictionary as the value of a key within a dictionary)
- Iterable
- Created by `dict()` or `{}`

```
art_galleries = {}

for name, zip_code in galleries:
    art_galleries[name] = zip_code
```

Printing in the loop

```
for name in sorted(art_galleries)[-5:]:
    print(name)
```

```
Zwirner David Gallery
Zwirner & Wirth
Zito Studio Gallery
Zetterquist Galleries
Zarre Andre Gallery
```

Safely finding by key

```
art_galleries['Louvre']
```

```
|---------------------------------------------------------------------
KeyError                                  Traceback (most recent call last)
<ipython-input-1-4f51c265f287> in <module>()
--> 1 art_galleries['Louvre']

KeyError: 'Louvre'
```

- Getting a value from a dictionary is done using the key as an index
- If you ask for a key that does not exist that will stop your program from running in a KeyError

Safely finding by key (cont.)

- `.get()` method allows you to safely access a key without error or exception handling
- If a key is not in the dictionary, `.get()` returns `None` by default or you can supply a value to return

```
art_galleries.get('Louvre', 'Not Found')
```

```
'Not Found'
```

```
art_galleries.get('Zarre Andre Gallery')
```

```
'10011'
```

Let's practice!

DATA TYPES IN PYTHON

Altering dictionaries

DATA TYPES IN PYTHON

Jason Myers
Instructor

Adding and extending dictionaries

- Assignment to add a new key/value to a dictionary
- `.update()` method to update a dictionary from another dictionary, tuples or keywords

```
print(galleries_10007)
```

```
{'Nyabinghi Africian Gift Shop': '(212) 566-3336'}
```

```
art_galleries['10007'] = galleries_10007
```

Updating a dictionary

```
galleries_11234 = [
  ('A J ARTS LTD', '(718) 763-5473'),
  ('Doug Meyer Fine Art', '(718) 375-8006'),
  ('Portrait Gallery', '(718) 377-8762')]
art_galleries['11234'].update(galleries_11234)
print(art_galleries['11234'])
```

```
{'Portrait Gallery': '(718) 377-8762',
'A J ARTS LTD': '(718) 763-5473',
'Doug Meyer Fine Art': '(718) 375-8006'}
```

Popping and deleting from dictionaries

- `del` instruction deletes a key/value
- `.pop()` method safely removes a key/value from a dictionary.

```
del art_galleries['11234']
galleries_10310 = art_galleries.pop('10310')
print(galleries_10310)
```

```
{'New Dorp Village Antiques Ltd': '(718) 815-2526'}
```

Let's practice!

DATA TYPES IN PYTHON

Pythonically using dictionaries

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Working with dictionaries more pythonically

- `.items()` method returns an object we can iterate over

```
for gallery, phone_num in art_galleries.items():
    print(gallery)
    print(phone_num)
```

```
'Miakey Art Gallery'
'(718) 686-0788'
'Morning Star Gallery Ltd'
'(212) 334-9330'}
'New York Art Expo Inc'
'(212) 363-8280'
```

Checking dictionaries for data

- `.get()` does a lot of work to check for a key
- `in` operator is much more efficient and clearer

```
'11234' in art_galleries
```

```
False
```

```
if '10010' in art_galleries:
    print('I found: %s' % art_galleries['10010'])
else:
    print('No galleries found.')
```

```
I found: {'Nyabinghi Africian Gift Shop': '(212) 566-3336'}
```

Let's practice!

DATA TYPES IN PYTHON

Mixed data types in dictionaries

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Working with nested dictionaries

```
art_galleries.keys()
```

```
dict_keys(['10021', '10013', '10001', '10009', '10011',
   ...: '10022', '10027', '10019', '11106', '10128'])
```

```
print(art_galleries['10027'])
```

```
{"Paige's Art Gallery": '(212) 531-1577',
 'Triple Candie': '(212) 865-0783',
 'Africart Motherland Inc': '(212) 368-6802',
 'Inner City Art Gallery Inc': '(212) 368-4941'}
```

- The `.keys()` method shows the keys for a given dictionary

Accessing nested data

```
art_galleries['10027']['Inner City Art Gallery Inc']
```

```
'(212) 368-4941'
```

- Common way to deal with repeating data structures
- Can be accessed using multiple indices or the `.get()` method

Let's practice!

DATA TYPES IN PYTHON

Numeric data types

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Built in numeric types

Integer

- Whole numbers
- Large values

```
int(12345678912345678 9)
```

```
12345678912345678 9
```

Float

- Fractional amounts (approximation)
- Scientific notation

```
float(12345678912345678 9)
```

```
1.2345678912345678e+17
```

Decimals

- Exact precision
- Currency operations

```
from decimal import Decimal
Decimal('12345678912345678 9')
```

```
Decimal('12345678912345678 9')
```

Printing floats

```
print(0.00001)
```

```
1e-05
```

```
print(f"{0.00001:f}")
```

```
0.000010
```

Printing floats

```
print(f"{0.0000001:f}")
```

```
0.000000
```

```
print(f"{0.0000001:.7f}")
```

```
0.0000001
```

Python division types

```
4/2
```

```
2.0
```

```
4//2
```

```
2
```

```
7//3
```

```
2
```

Let's practice!

DATA TYPES IN PYTHON

Booleans - the logical data type

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Booleans as a data type

- `True`
- `False`

Notice the capitalization as this can trip you up when switching between Python and other languages.

```
out_of_cookies = True
if out_of_cookies:
    print("Run to the store NOW!")
```

```
Run to the store NOW!
```

Truthy and Falsey

- Truthy values are ones that will return true
- Falsey values will evaluate to false

```
apples=2
if apples:
    print("We have apples.")
```

```
"We have apples."
```

```
apples=0
if apple:
    print('We have apples.')
```

Truthy and Falsey

Truthy

- `1`
- `"Cookies"`
- `["Cake", "Pie"]`
- `{"key": "value"}`

Falsey

- `0`
- `""`
- `[]`
- `{}`
- `None`

Operators - a boolean evaluation context

```
cookie_qty == 3
```

- `==` equal to
- `!=` not equal to
- `<` less than
- `<=` less than or equal to
- `>` greater than
- `>=` greater than or equal to

Floats are approximately an issue

```
x = 0.1 + 1.1
x == 1.2
```

```
False
```

```
print(x)
```

```
1.2000000000000002
```

Be careful with equality comparisons of floats!

Let's practice!

DATA TYPES IN PYTHON

Sets (unordered data with optimized logic operations)

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Set

- Unique
- Unordered
- Mutable
- Python's implementation of Set Theory from Mathematics

Creating sets

- Sets are created from a list

```
cookies_eaten_today = ['chocolate chip', 'peanut butter',
   ...: 'chocolate chip', 'oatmeal cream', 'chocolate chip']
types_of_cookies_eaten = set(cookies_eaten_today)
print(types_of_cookies_eaten)
```

```
set(['chocolate chip', 'oatmeal cream', 'peanut butter'])
```

Modifying sets

- `.add()` adds single elements

```
types_of_cookies_eaten.add('biscotti')

types_of_cookies_eaten.add('chocolate chip')

print(types_of_cookies_eaten)
```

```
set(['chocolate chip', 'oatmeal cream', 'peanut butter', 'biscotti'])
```

Updating sets

- `.update()` merges in another set or list

```
cookies_hugo_ate = ['chocolate chip', 'anzac']

types_of_cookies_eaten.update(cookies_hugo_ate)

print(types_of_cookies_eaten)
```

```
set(['chocolate chip', 'anzac', 'oatmeal cream',  'peanut
```

Removing data from sets

- `.discard()` safely removes an element from the set by value
- `.pop()` removes and returns an arbitrary element from the set (KeyError when empty)

```
types_of_cookies_eaten.discard('biscotti')
print(types_of_cookies_eaten)
```

```
set(['chocolate chip', 'anzac', 'oatmeal cream', 'peanut butter'])
```

```
types_of_cookies_eaten.pop()
types_of_cookies_eaten.pop()
```

```
'chocolate chip'
'anzac'
```

Set operations - similarities

- `.union()` set method returns a set of all the names (`or`)
- `.intersection()` method identifies overlapping data (`and`)

```
cookies_jason_ate = set(['chocolate chip', 'oatmeal cream',
'peanut butter'])
cookies_hugo_ate = set(['chocolate chip', 'anzac'])
cookies_jason_ate.union(cookies_hugo_ate)
```

```
set(['chocolate chip', 'anzac', 'oatmeal cream', 'peanut butter'])
```

```
cookies_jason_ate.intersection(cookies_hugo_ate)
```

```
set(['chocolate chip'])
```

Set operations - differences

- `.difference()` method identifies data present in the set on which the method was used that is not in the arguments (`-`)
- Target is important!

```
cookies_jason_ate.difference(cookies_hugo_ate)
```

```
set(['oatmeal cream', 'peanut butter'])
```

```
cookies_hugo_ate.difference(cookies_jason_ate)
```

```
set(['anzac'])
```

Let's practice!

DATA TYPES IN PYTHON

Counting made easy

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Collections Module

- Part of Standard Library
- Advanced data containers

Counter

- Special dictionary used for counting data, measuring frequency

```
from collections import Counter
nyc_eatery_count_by_types = Counter(nyc_eatery_types)
print(nyc_eatery_count_by_type)
```

```
Counter({'Mobile Food Truck': 114, 'Food Cart': 74, 'Snack Bar': 24,
'Specialty Cart': 18, 'Restaurant': 15, 'Fruit & Vegetable Cart': 4})
```

```
print(nyc_eatery_count_by_types['Restaurant'])
```

```
15
```

Counter to find the most common

- `.most_common()` method returns the counter values in descending order

```
print(nyc_eatery_count_by_types.most_common(3))
```

```
[('Mobile Food Truck', 114), ('Food Cart', 74), ('Snack Bar', 24)]
```

Let's practice!

DATA TYPES IN PYTHON

Dictionaries of unknown structure - defaultdict

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Dictionary Handling

```
for park_id, name in nyc_eateries_parks:
    if park_id not in eateries_by_park:
        eateries_by_park[park_id] = []
    eateries_by_park[park_id].append(name)
print(eateries_by_park['M010'])
```

```
{'MOHAMMAD MATIN','PRODUCTS CORP.', 'Loeb Boathouse Restaurant',
'Nandita Inc.', 'SALIM AHAMED', 'THE NY PICNIC COMPANY',
'THE NEW YORK PICNIC COMPANY, INC.', 'NANDITA, INC.',
'JANANI FOOD SERVICE, INC.'}
```

Using defaultdict

- Pass it a default type that every key will have even if it doesn't currently exist
- Works exactly like a dictionary

```
from collections import defaultdict
eateries_by_park = defaultdict(list)
for park_id, name in nyc_eateries_parks:
    eateries_by_park[park_id].append(name)
print(eateries_by_park['M010'])
```

```
{'MOHAMMAD MATIN','PRODUCTS CORP.', 'Loeb Boathouse Restaurant',
'Nandita Inc.', 'SALIM AHAMED', 'THE NY PICNIC COMPANY',
'THE NEW YORK PICNIC COMPANY, INC.', 'NANDITA, INC.', ...}
```

Using defaultdict

```
from collections import defaultdict
eatery_contact_types = defaultdict(int)
for eatery in nyc_eateries:
    if eatery.get('phone'):
        eatery_contact_types['phones'] += 1
    if eatery.get('website'):
        eatery_contact_types['websites'] += 1
print(eatery_contact_types)
```

```
defaultdict(<class 'int'>, {'phones': 28, 'websites': 31})
```

Let's practice!

DATA TYPES IN PYTHON

namedtuple

DATA TYPES IN PYTHON

Jason Myers
Instructor

What is a namedtuple?

- A tuple where each position (column) has a name
- Ensure each one has the same properties
- Alternative to a `pandas` DataFrame row

Creating a namedtuple

- Pass a name and a list of fields

```
from collections import namedtuple
Eatery = namedtuple('Eatery', ['name', 'location', 'park_id',
   ...: 'type_name'])
eateries = []
for eatery in nyc_eateries:
    details = Eatery(eatery['name'],
                     eatery['location'],
                     eatery['park_id'],
                     eatery['type_name'])
    eateries.append(details)
```

Print the first element

```
print(eateries[0])
```

```
Eatery(name='Mapes Avenue Ballfields Mobile Food Truck',
location='Prospect Avenue, E. 181st Street',
park_id='X289', type_name='Mobile Food Truck')
```

Leveraging namedtuples

- Each field is available as an attribute of the namedtuple

```
for eatery in eateries[:3]:
    print(eatery.name)
    print(eatery.park_id)
    print(eatery.location)
```

```
Mapes Avenue Ballfields Mobile Food Truck
X289
Prospect Avenue, E. 181st Street

Claremont Park Mobile Food Truck
X008
East 172 Street between Teller & Morris avenues ...
```

Let's practice!

DATA TYPES IN PYTHON

Dataclasses

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Why use dataclasses

- Support for default values
- Custom representations of the objects
- Easy tuple or a dictionary conversion
- Custom properties
- Frozen instances

Looking at our first dataclass

```
from dataclasses import dataclass
```

```
@dataclass
class Cookie:
    name: str
    quantity: int = 0
```

```
chocolate_chip = Cookie("chocolate chip", 13)
print(chocolate_chip.name)
print(chocolate_chip.quantity)
```

```
chocolate chip
13
```

Easy tuple or a dictionary conversion

```
from dataclasses import asdict, astuple


ginger_molasses = Cookie("ginger molasses", 8)
asdict(ginger_molasses)
```

```
{'name': 'ginger molasses', 'quantity': 8}
```

```
astuple(ginger_molasses)
```

```
('ginger molasses', 8)
```

Custom properties

```
from decimal import Decimal



@dataclass
class Cookie:
    name: str
    cost: Decimal
    quantity: int
```

```
    @property
    def value_of_goods(self):
        return int(self.quantity) * self.cost
```

Using custom properties

```
peanut = Cookie("peanut butter", Decimal("1.2"), 8)


peanut.value_of_goods
```

```
Decimal('9.6')
```

Frozen instances

```
@dataclass(frozen=True)
class Cookie:
    name: str
    quantity: int = 0

c = Cookie("chocolate chip", 10)
```

```
c.quantity = 15
```

```
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
  File "<string>", line 4, in __setattr__
dataclasses.FrozenInstanceError: cannot assign to field 'quantity'
```

Let's practice!

DATA TYPES IN PYTHON

Wrap-up

DATA TYPES IN PYTHON

Jason Myers
Instructor

datacamp

Sequence data types

- Lists

```
['Chocolate Chip', 'Peanut Butter']
```

- Tuples

```
('Sugar', 'Eggs')
```

- Strings

```
'Cookies are wonderful'
```

Dictionaries

- Safely adding and removing

```
squirrels_by_park.pop("City Hall Park", {})
```

- Unpacking items

```
for field, value in squirrels_by_park.items():
```

- Handling nested data

```
for park in squirrels_by_park:
    print(squirrels_by_park[park].get('color', 'N/A'))
```

Numeric and logical types

- Integers `int(1)`
- Floats `float(1.333333334)`
- Decimals `Decimal(5.50)`
- Booleans `True or False`
- Sets `{'Anzac', 'Oatmeal Raisin'}`

Complex data types

- Counters

```
Counter(nyc_eatery_types)
```

- Defaultdicts

```
eateries_by_park = defaultdict(list)
for park_id, name in nyc_eateries_parks:
  eateries_by_park[park_id].append(name)
```

- Namedtuples

```
namedtuple('Worm', ['species', 'sex', 'mass'])
```

Complex data types

- Dataclasses

```
@dataclass
class WeightEntry:
    species: str
    flipper_length: int
    body_mass: int
    sex: str
```

Congratulations!

DATA TYPES IN PYTHON